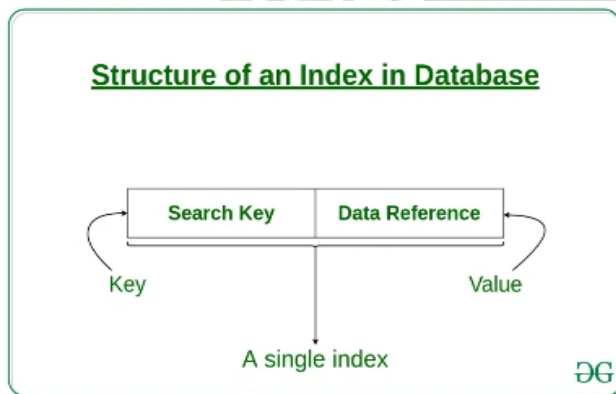


UNIT-V

5.2 Indexing in DBMS

Indexing in DBMS is used to speed up data retrieval by minimizing disk scans. Instead of searching through all rows, the DBMS uses index structures to quickly locate data using key values.

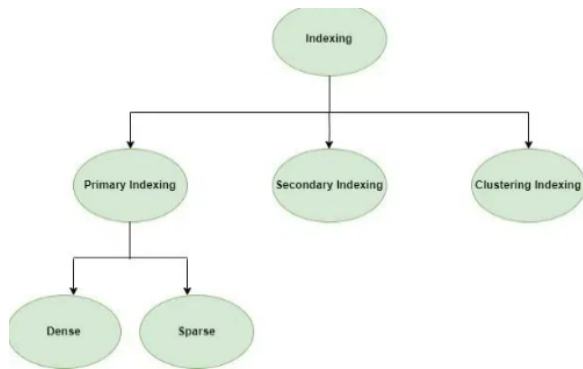
When an index is created, it stores sorted key values and pointers to actual data rows. This reduces the number of disk accesses, improving performance especially on large datasets.



Attributes of Indexing

Several Important attributes of indexing affect the performance and efficiency of database operations:

1. **Access Types:** This refers to the type of access such as value-based search, range access, etc.
2. **Access Time:** It refers to the time needed to find a particular data element or set of elements.
3. **Insertion Time:** It refers to the time taken to find the appropriate space and insert new data.
4. **Deletion Time:** Time taken to find an item and delete it as well as update the index structure.
5. **Space Overhead:** It refers to the additional space required by the index.



File Organization in Indexing

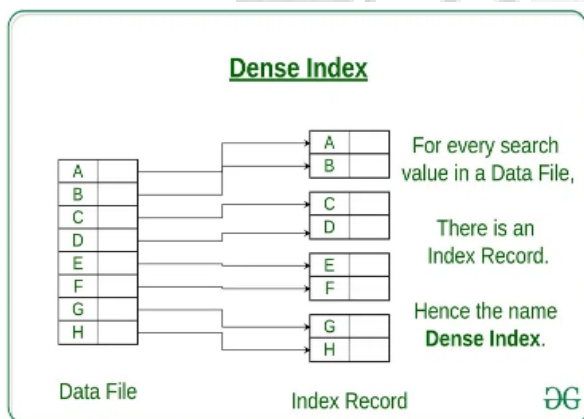
File organization refers to how data and indexes are physically stored in memory or on disk. The following are the common types of file organizations used in indexing:

1. Sequential (Ordered) File Organization

In this type of organization, the indices are based on a sorted ordering of the values. These are generally fast and a more traditional type of storing mechanism. These Ordered or Sequential file organizations might store the data in a dense or sparse format.

i. Dense Index: Every search key value in the data file corresponds to an index record. This method ensures that each key value has a reference to its data location.

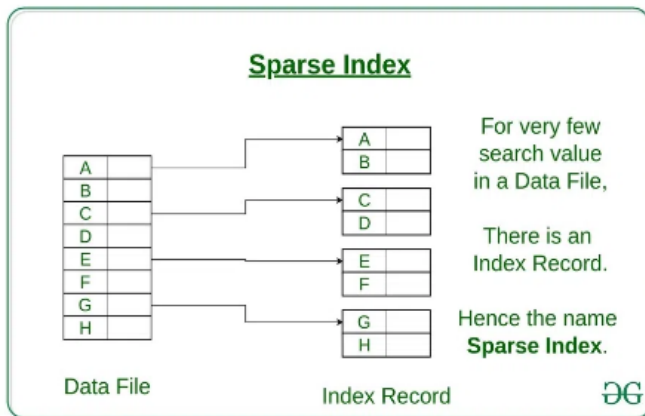
Example: If a table contains multiple entries for the same key, a dense index ensures that each key value has its own index record.



ii. Sparse Index: The index record appears only for a few items in the data file. Each item points to a block as shown. To locate a record, we find the index record with the largest search key value less than or equal to the search key value we are looking for.

Access Method: To locate a record, we find the index record with the largest key value less than or equal to the search key, and then follow the pointers sequentially.

Access Cost $= \log_2(n) + 1 = \log_2(n) + 1$, where n is the number of blocks involved in the index file.



2. Hash File Organization

Uses a hash function to map keys to buckets.

- Offers fast access for exact-match queries.
- Not suitable for range queries.

Types of Indexing Methods

There are different types of indexing techniques, each optimized for specific use cases.

1. Clustered Indexing

Clustered Indexing stores related records together in the same file, reducing search time and improving performance, especially for join operations. Data is stored in sorted order based on a key (often a **non-primary key**) to group similar records, like students by semester. If the indexed column isn't unique, multiple columns can be combined to form a unique key. This makes data retrieval faster by keeping related records close and allowing quicker access through the index.

INDEX FILE		Data Blocks in Memory				
SEMESTER	INDEX ADDRESS					
1		100	Joseph	Alaiedon Township	20	200
2		101				
3						
4		110	Allen	Fraser Township	20	200
5		111				
		120	Chris	Clinton Township	21	200
		121				
		200	Patty	Troy	22	205
		201				
		210	Jack	Fraser Township	21	202
		211				
		300				

2. Primary Indexing

This is a type of Clustered Indexing wherein the data is sorted according to the search key and the **primary key** of the database table is used to create the index. It is a default format of indexing where it induces sequential file organization. As primary keys

are **unique** and are stored in a sorted manner, the performance of the **searching operation** is quite efficient.

Key Features: The data is stored in sequential order, making searches faster and more efficient.

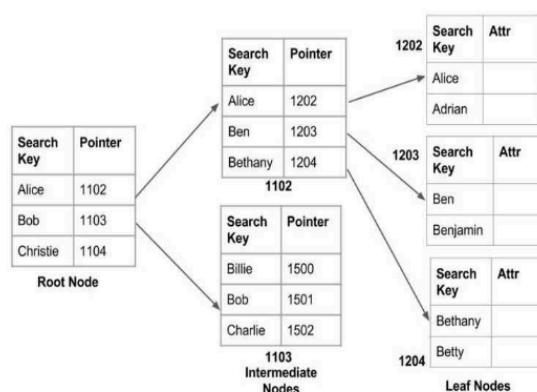
3. Non-clustered or Secondary Indexing

A non-clustered index just tells us where the data lies, i.e. it gives us a list of virtual pointers or references to the location where the data is actually stored. Data is not physically stored in the order of the index. Instead, data is present in leaf nodes.

Example:

The contents page of a book. Each entry gives us the page number or location of the information stored. The actual data here (information on each page of the book) is not organized but we have an ordered reference (contents page) to where the data points actually lie. We can have only dense ordering in the non-clustered index as sparse ordering is not possible because data is not physically organized accordingly.

It requires more time as compared to the clustered index because some amount of extra work is done in order to extract the data by further following the pointer. In the case of a clustered index, data is directly present in front of the index.

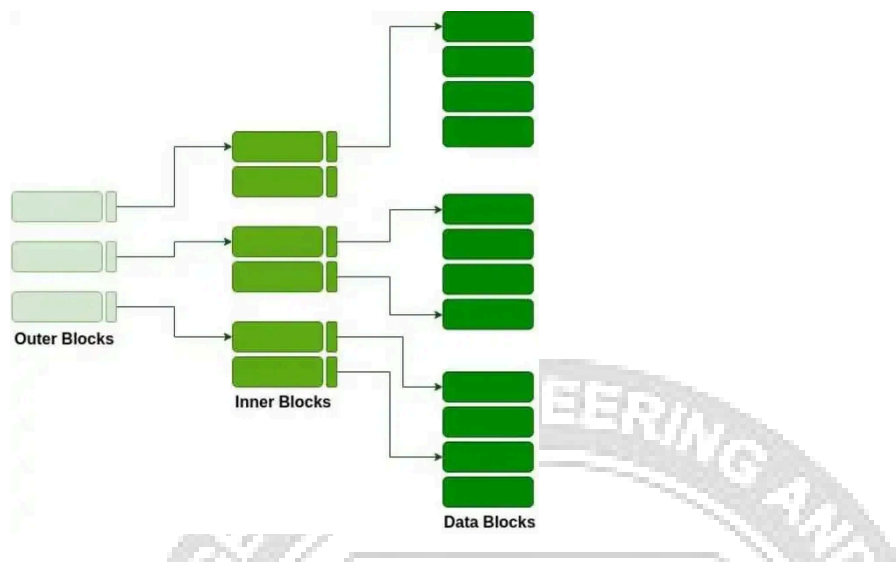


Non clustered index

4. Multilevel Indexing

With the growth of the size of the database, indices also grow. As the index is stored in the **main memory**, a single-level index might become too large a size to store with **multiple disk accesses**. The multilevel indexing segregates the main block into various smaller blocks so that the same can be stored in a single block.

The outer blocks are divided into **inner blocks** which in turn are pointed to the data blocks. This can be easily stored in the main memory with fewer overheads. This hierarchical approach reduces memory overhead and speeds up query execution.



Advantages of Indexing

- **Faster Queries:** Indexes allow quick search of rows matching specific values, speeding up data retrieval.
- **Efficient Access:** Reduces disk I/O by keeping frequently accessed data in memory.
- **Improved Sorting:** Speeds up sorting by indexing the relevant columns.
- **Consistent Performance:** Maintains query speed even as data grows.
- **Data Integrity:** Ensures uniqueness in columns indexed as unique, preventing duplicate entries.

Disadvantages of Indexing

While indexing offers many advantages, it also comes with certain trade-offs:

- **Increased Storage Space:** Indexes require additional storage. Depending on the size of the data, this can significantly increase the overall storage requirements.
- **Increased Maintenance Overhead:** Indexes must be updated whenever data is inserted, deleted, or modified, which can slow down these operations.
- **Slower Insert/Update Operations:** Since indexes must be maintained and updated, inserting or updating data takes longer than in a non-indexed database.
- **Complexity in Choosing the Right Index:** Determining the appropriate indexing strategy for a particular dataset can be challenging and requires an understanding of query patterns and access behaviors.

Features of Indexing

Several key features define the indexing process in databases:

- **Efficient Data Structures:** Indexes use efficient data structures like B-trees, B+ trees, and hash tables to enable fast data retrieval.
- **Periodic Index Maintenance:** Indexes need to be periodically maintained, especially when the underlying data changes frequently. Maintenance tasks include updating, rebuilding, or removing obsolete indexes.
- **Query Optimization:** Indexes play a critical role in query optimization. The DBMS query optimizer uses indexes to determine the most efficient execution plan for a query.
- **Handling Fragmentation:** Index fragmentation can reduce the effectiveness of an index. Regular defragmentation can help maintain optimal performance.